ORGCHAT

Modèle relationnel – Schéma des tables et relations

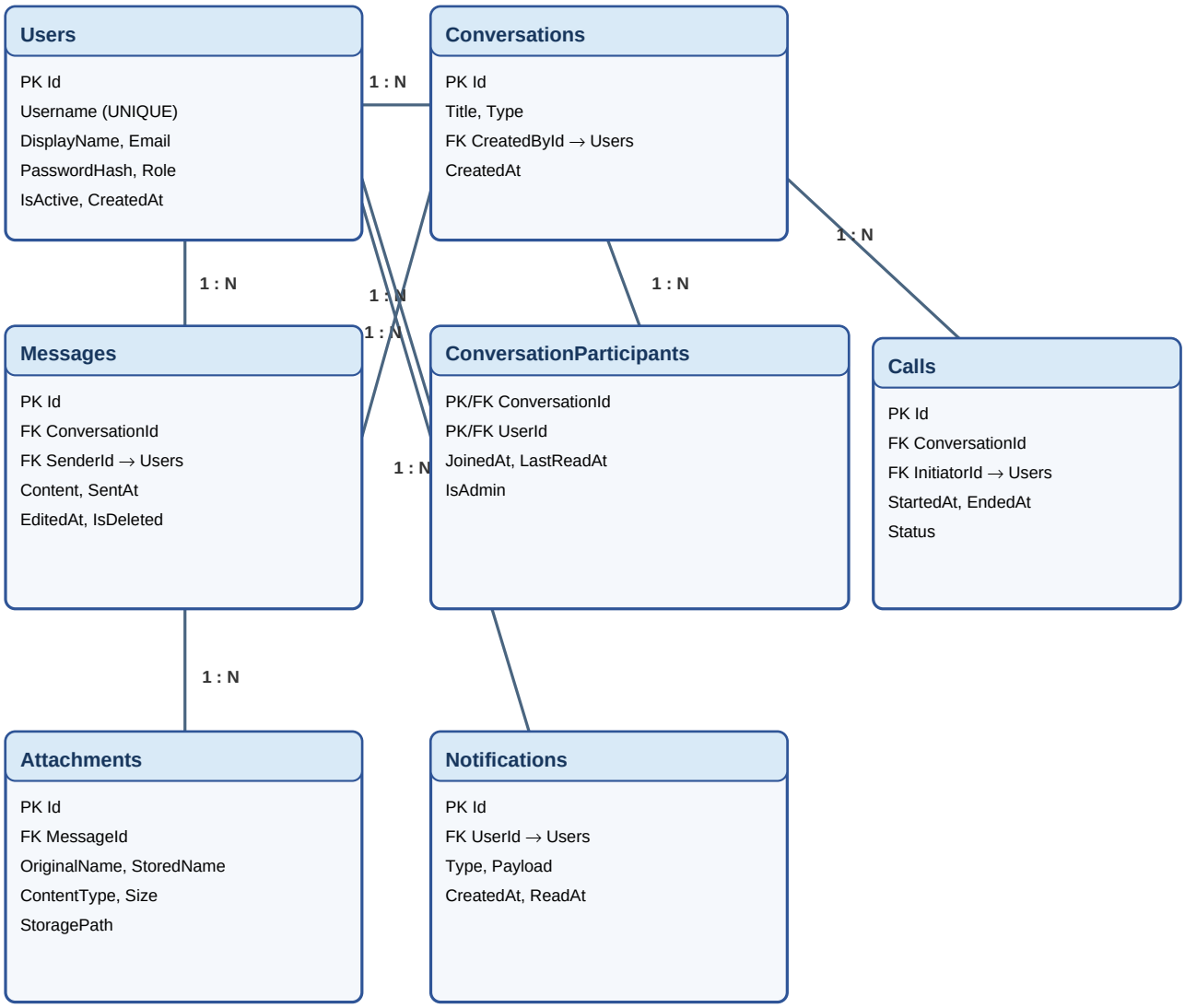
Projet académique : application de messagerie réseau en C# / .NET, ASP.NET Core, SignalR et SQLite

1. Objectif du document

Ce document sert de référence au programmeur chargé de la base de données. Il présente les entités, leurs clés, les relations entre tables, les cardinalités et les règles d'intégrité à appliquer dans SQLite. Le modèle est une proposition de départ à valider collectivement avant les migrations EF Core.

Décision d'architecture : le fichier SQLite est conservé sur la machine serveur OrgChat. Seul le backend ASP.NET Core accède à ce fichier. Les clients Windows Forms communiquent avec le backend par HTTP(S) et SignalR ; ils n'ouvrent jamais directement le fichier .db.

2. Vue d'ensemble des relations



Lecture du diagramme : 1 : N signifie qu'une ligne du côté « 1 » peut être associée à plusieurs lignes du côté « N ». La table ConversationParticipants matérialise la relation plusieurs-à-plusieurs entre Users et Conversations.

3. Dictionnaire des tables

Les types ci-dessous sont indicatifs et adaptés à SQLite. Avec EF Core, les identifiants peuvent être INTEGER PRIMARY KEY AUTOINCREMENT ou des GUID stockés en TEXT. Choisir une stratégie unique pour tout le projet.

Users

Champ	Type	Contrainte	Description
Id	INTEGER	PK	Identifiant utilisateur
Username	TEXT	UNIQUE, NOT NULL	Identifiant de connexion
DisplayName	TEXT	NOT NULL	Nom affiché
Email	TEXT	NULL / UNIQUE	Adresse e-mail si utilisée
PasswordHash	TEXT	NOT NULL	Hash du mot de passe, jamais le mot de passe
Role	TEXT	NOT NULL	Utilisateur / Administrateur
IsActive	INTEGER	NOT NULL DEFAULT 1	Compte actif ou désactivé
CreatedAt	TEXT	NOT NULL	Date UTC ISO-8601

Conversations

Champ	Type	Contrainte	Description
Id	INTEGER	PK	Identifiant conversation
Title	TEXT	NULL	Nom du groupe ; facultatif en privé
Type	TEXT	NOT NULL	Private ou Group
CreatedById	INTEGER	FK → Users.Id	Créateur
CreatedAt	TEXT	NOT NULL	Date de création UTC

ConversationParticipants

Champ	Type	Contrainte	Description
ConversationId	INTEGER	PK, FK → Conversations.Id	Conversation concernée
UserId	INTEGER	PK, FK → Users.Id	Membre
JoinedAt	TEXT	NOT NULL	Date d'ajout
LastReadAt	TEXT	NULL	Dernière date de lecture
IsAdmin	INTEGER	NOT NULL DEFAULT 0	Responsable du groupe

Messages

Champ	Type	Contrainte	Description
Id	INTEGER	PK	Identifiant message
ConversationId	INTEGER	FK → Conversations.Id	Conversation
SenderId	INTEGER	FK → Users.Id	Expéditeur
Content	TEXT	NULL si pièce jointe seule	Texte du message
SentAt	TEXT	NOT NULL	Date serveur UTC
EditedAt	TEXT	NULL	Date de modification
IsDeleted	INTEGER	NOT NULL DEFAULT 0	Suppression logique

Attachments

Champ	Type	Contrainte	Description
Id	INTEGER	PK	Identifiant fichier
MessageId	INTEGER	FK → Messages.Id	Message associé
OriginalName	TEXT	NOT NULL	Nom présenté à l'utilisateur
StoredName	TEXT	NOT NULL, UNIQUE	Nom généré côté serveur
ContentType	TEXT	NOT NULL	Type MIME détecté/validé

Champ	Type	Contrainte	Description
Size	INTEGER	NOT NULL	Taille en octets
StoragePath	TEXT	NOT NULL	Chemin logique interne

Notifications

Champ	Type	Contrainte	Description
Id	INTEGER	PK	Identifiant notification
UserId	INTEGER	FK → Users.Id	Destinataire
Type	TEXT	NOT NULL	Nouveau message, ajout groupe, etc.
Payload	TEXT	NULL	Données utiles, sans secret
CreatedAt	TEXT	NOT NULL	Date UTC
ReadAt	TEXT	NULL	Date de lecture

Calls

Champ	Type	Contrainte	Description
Id	INTEGER	PK	Identifiant appel
ConversationId	INTEGER	FK → Conversations.Id	Conversation concernée
InitiatorId	INTEGER	FK → Users.Id	Utilisateur initiateur
StartedAt	TEXT	NOT NULL	Début
EndedAt	TEXT	NULL	Fin
Status	TEXT	NOT NULL	Ringing, Accepted, Rejected, Ended, Failed

4. Relations et cardinalités

Relation	Cardinalité	Interprétation
Users → Conversations	1 à N	Un utilisateur peut créer plusieurs conversations. Chaque conversation a un c
Users ↔ Conversations	N à N via ConversationParticipants	Un utilisateur participe à plusieurs conversations ; une conversation contient p
Conversations → Messages	1 à N	Une conversation contient zéro ou plusieurs messages ; chaque message app
Users → Messages	1 à N	Un utilisateur peut envoyer plusieurs messages ; chaque message a un expé
Messages → Attachments	1 à N	Un message peut contenir zéro ou plusieurs pièces jointes. Une pièce jointe a
Users → Notifications	1 à N	Un utilisateur reçoit plusieurs notifications.
Conversations → Calls	1 à N	Une conversation peut avoir plusieurs sessions d'appel enregistrées.
Users → Calls	1 à N	Un utilisateur peut initier plusieurs appels.

5. Contraintes d'intégrité recommandées

• Activer les clés étrangères SQLite à chaque connexion : PRAGMA foreign_keys = ON. Avec EF Core, vérifier que la connexion et les migrations appliquent bien ce paramètre.

• Users.Username doit être unique et non nul. Email peut être unique si l'inscription impose une adresse.

• ConversationParticipants utilise une clé primaire composite (ConversationId, UserId) pour empêcher les doublons de membres.

• Messages.ConversationId et Messages.SenderId sont obligatoires. Avant insertion, le backend vérifie que SenderId est bien participant de la conversation.

• Attachments.StoredName doit être unique. Le fichier physique doit être conservé hors du dossier public de l'application.

• Ajouter des index sur Messages(ConversationId, SentAt), ConversationParticipants(UserId), Notifications(UserId, ReadAt) et les clés étrangères utilisées dans les recherches.

• Utiliser des dates UTC, par exemple au format ISO-8601, de façon cohérente.

• Définir explicitement les règles ON DELETE : éviter les suppressions en cascade non maîtrisées. Pour les utilisateurs, préférer une désactivation ; pour les messages, envisager une suppression logique.

6. Exemple de schéma SQL de départ

Exemple pédagogique simplifié. Les migrations EF Core doivent rester la source officielle du schéma une fois le projet démarré.

```
PRAGMA foreign_keys = ON; CREATE TABLE Users ( Id INTEGER PRIMARY KEY AUTOINCREMENT, Username TEXT NOT NULL
UNIQUE, DisplayName TEXT NOT NULL, Email TEXT UNIQUE, PasswordHash TEXT NOT NULL, Role TEXT NOT NULL DEFAULT
'User', IsActive INTEGER NOT NULL DEFAULT 1, CreatedAt TEXT NOT NULL ); CREATE TABLE Conversations ( Id
INTEGER PRIMARY KEY AUTOINCREMENT, Title TEXT, Type TEXT NOT NULL CHECK (Type IN ('Private', 'Group')),
CreatedById INTEGER NOT NULL, CreatedAt TEXT NOT NULL, FOREIGN KEY (CreatedById) REFERENCES Users(Id) );
CREATE TABLE ConversationParticipants ( ConversationId INTEGER NOT NULL, UserId INTEGER NOT NULL, JoinedAt
TEXT NOT NULL, LastReadAt TEXT, IsAdmin INTEGER NOT NULL DEFAULT 0, PRIMARY KEY (ConversationId, UserId),
FOREIGN KEY (ConversationId) REFERENCES Conversations(Id), FOREIGN KEY (UserId) REFERENCES Users(Id) );
CREATE TABLE Messages ( Id INTEGER PRIMARY KEY AUTOINCREMENT, ConversationId INTEGER NOT NULL, SenderId
INTEGER NOT NULL, Content TEXT, SentAt TEXT NOT NULL, EditedAt TEXT, IsDeleted INTEGER NOT NULL DEFAULT 0,
FOREIGN KEY (ConversationId) REFERENCES Conversations(Id), FOREIGN KEY (SenderId) REFERENCES Users(Id) );
CREATE TABLE Attachments ( Id INTEGER PRIMARY KEY AUTOINCREMENT, MessageId INTEGER NOT NULL, OriginalName
TEXT NOT NULL, StoredName TEXT NOT NULL UNIQUE, ContentType TEXT NOT NULL, Size INTEGER NOT NULL CHECK (Size
>= 0), StoragePath TEXT NOT NULL, FOREIGN KEY (MessageId) REFERENCES Messages(Id) ); CREATE INDEX
IX_Messages_Conversation_SentAt ON Messages(ConversationId, SentAt); CREATE INDEX IX_Participants_User ON
ConversationParticipants(UserId); CREATE INDEX IX_Notifications_User_ReadAt ON Notifications(UserId, ReadAt);
```

7. Conseils pratiques pour le programmeur SQLite

1) Valider les cardinalités et règles de suppression avec le backend avant de créer les migrations. 2) Centraliser le DbContext et les configurations EF Core dans OrgChat.Infrastructure. 3) Ne pas créer manuellement un deuxième schéma concurrent à celui des migrations. 4) Prévoir des données de démonstration non sensibles. 5) Tester les migrations sur une base vide et sur une copie de la base existante. 6) Sauvegarder ensemble la base et les pièces jointes afin de préserver les

références.

8. Périmètre optionnel à décider

Les tables Calls sont utiles si l'équipe conserve l'historique des appels audio. Si les appels ne sont pas réalisés dans la première version, cette table peut rester une extension future. De même, une table MessageReceipts dédiée peut être ajoutée si le projet exige un accusé de lecture individuel par message et par utilisateur ; le champ LastReadAt dans ConversationParticipants suffit pour un indicateur simple « lu jusqu'à ».

Fin du document – OrgChat / Modèle relationnel v1.0